Symmetry-Aware Transformer Training for Automated Planning

One-line Summary

Planning problems contain lots of meaningless object-name symmetry. Standard transformers fail because they accidentally memorise names and positions. We fix this by training transformers to treat renamed versions of the same problem equivalently, while also redesigning parts of the architecture to respect these symmetries.

Problem

Even though planning is generating a sequence of action which is just like seq2seq learning why do transformers fail?
Reason : ambiguity in planning representation i.e equivalent encoding and transformers do not know inductive bias.
Object names are just variables. Renaming objects does not change the underlying problem, but standard transformers treat different names as different inputs because they rely heavily on token identities and positional information.

Core Idea

Instead of teaching a transformer that every possible renaming of a planning problem is a different example, train it to recognise that they are the same problem. The paper achieves this through symmetry-aware contrastive learning, which forces renamed versions of the same planning task to produce similar internal representations and attention patterns. This allows the model to focus on relational structure rather than arbitrary object names. They also completely remove positional encoding.

The central claim of the paper is that vanilla transformers waste learning capacity on arbitrary object names and ordering information, while symmetry-aware training encourages them to focus on the underlying planning structure instead.

Key Concepts

Inductive Bias: Prior assumptions built into a model that help it learn certain patterns more easily. Classical planning methods and GNNs naturally respect relational structure, while transformers largely have to learn it from data.

Symmetry: Multiple representations can describe the same planning problem. For example, renaming all objects does not change the actual task.

Equivariance: If object names are changed, the model's internal reasoning should change consistently while preserving the underlying structure.

Contrastive Learning: A training technique that encourages similar inputs to have similar internal representations. Here, renamed versions of the same planning problem are treated as similar examples.

NoPE (No Positional Encoding): Removing positional encodings so that the model does not learn artificial ordering information from planning states.

Method pipeline

1. **Architecture Redesign**
   - Use an encoder-only architecture for heuristic prediction and an encoder-decoder architecture for plan generation.
   - Remove positional encodings to avoid learning meaningless ordering information.
2. **Atom Embeddings**
   - Instead of representing a fact as multiple tokens, each planning atom is converted into a single embedding.
   - This preserves relational structure while avoiding dependence on token order.
3. **Symmetry-Aware Contrastive Training**
   - Generate two versions of the same planning problem with different object names.
   - Force the model to process both versions similarly.
4. **Attention Loss**
   - Encourages identical attention patterns across renamed problems.
5. **Hidden-State Loss**
   - Encourages similar internal representations across renamed problems.
6. **Prediction Loss**
   - Standard loss for plan generation or heuristic prediction.

The final training objective combines prediction loss, attention loss, and hidden-state loss.

Results

- Significantly better extrapolation than PlanGPT.
- Encoder-decoder model achieved highest coverage.
- Generated plans were often optimal or near-optimal.
- Contrastive loss improved training stability.
- Logistics remained challenging for all models.

Weaknesses

- Still fails to extrapolate in Logistics.
- Greedy decoding performs worse than search-based decoding.
- True algorithmic generalisation is not fully achieved.
- Fixed vocabulary limits the number of objects that can be handled.

My thoughts

- Equivalent representation of the same planning problem is being handled efficiently through the 2 version planning
- Why concatenation and not adding up the vectors ?
- The reason behind the failure in Logistics is not fully clear.
- And also why the first dk dimensions of the hidden state it being used and not all.

Connections

Heuristic Search
STRIPS